PES UNIVERSITY

Department of Computer Science & Engineering

Object Oriented Analysis and Design

UE23CS352B

Assignment 9 Submission

Name of the Student	Pranav Hemanth
SRN	PES1UG23CS433
Section	G
Department	CSE
Campus	RR

Object Oriented Analysis and Design (UE23CS352B) - Lab 9

Problems on Notification System and Planner

UE23CS352B

Problem Statement

1. You are developing a multi-channel notification system for a large enterprise similar to Amazon Web Services. The system must support Email, SMS, and Push notifications.
   Creating separate service objects repeatedly leads to memory overhead and inconsis- tent configurations.
   Design a solution where a Factory Pattern dynamically selects the notification type at runtime, while each notification service is implemented as a Singleton to ensure only one instance exists per type.

Source files list:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 q1 %ls
MailNotifService.class  Main.java               NotifService.class      PushNotifService.java
MailNotifService.java   NotifFactory.class      NotifService.java       SMSNotifService.class
Main.class              NotifFactory.java       PushNotifService.class  SMSNotifService.java
pranavhemanth@Pranavs-MacBook-Pro-M3 q1 %tree
.
├── MailNotifService.class
├── MailNotifService.java
├── Main.class
├── Main.java
├── NotifFactory.class
├── NotifFactory.java
├── NotifService.class
├── NotifService.java
├── PushNotifService.class
├── PushNotifService.java
├── SMSNotifService.class
└── SMSNotifService.java

1 directory, 12 files
pranavhemanth@Pranavs-MacBook-Pro-M3 q1 %
```

Project structure for Q1 showing all source files: NotifService interface, three Singleton service classes (Mail/SMS/Push), NotifFactory, and Main.

NotifService.java

```
Lab9 > q1 > J NotifService.java > ...
1   package Lab9.q1;
2
3   // Abstract product interface
4   public interface NotifService {
5       void send(String recipient, String message);
6   }
7
```

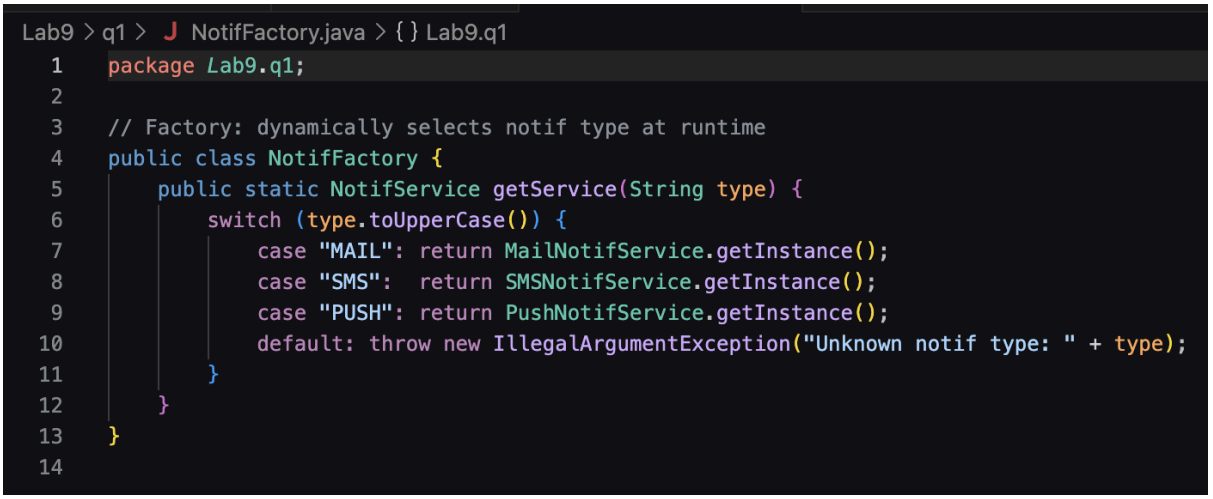
NotifService is the abstract product interface defining the contract send(recipient, message) for all notification types.

MailNotifService.java



MailNotifService implements NotifService as a Singleton: private constructor prevents external instantiation, getInstance() returns the single shared instance.

NotifFactory.java



SMSNotifService.java

Lab9 > q1 > J SMSNotifService.java > { } Lab9.q1

```java
package Lab9.q1;

// Singleton: SMS Notif Service
public class SMSNotifService implements NotifService {
    private static SMSNotifService instance;

    private SMSNotifService() {}

    public static SMSNotifService getInstance() {
        if (instance == null) {
            instance = new SMSNotifService();
        }
        return instance;
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("[SMS] To: " + recipient + " | Message: " + message);
    }
}

```

SMSNotifService implements NotifService as a Singleton. It follows the same private constructor + getInstance() pattern, ensuring only one SMS service instance exists throughout the application.

PushNotifService.java

Lab9 > q1 > PushNotifService.java > { } Lab9.q1

```java
package Lab9.q1;

// Singleton: Push Notif Service
public class PushNotifService implements NotifService {
    private static PushNotifService instance;

    private PushNotifService() {}

    public static PushNotifService getInstance() {
        if (instance == null) {
            instance = new PushNotifService();
        }
        return instance;
    }

    @Override
    public void send(String recipient, String message) {
        System.out.println("[PUSH] To: " + recipient + " | Message: " + message);
    }
}

```

PushNotifService implements NotifService as a Singleton for push notifications. The Factory returns this single shared instance every time PUSH type is requested.

Main.java

Lab9 > q1 > Main.java > { } Lab9.q1

```java
package Lab9.q1;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        // Factory selects service at runtime; each service is a Singleton
        NotifService mailService = NotifFactory.getService(type: "MAIL");
        mailService.send(recipient: "alice@example.com", message: "Your order has been placed.");

        NotifService smsService = NotifFactory.getService(type: "SMS");
        smsService.send(recipient: "+919876543210", message: "OTP: 482910");

        NotifService pushService = NotifFactory.getService(type: "PUSH");
        pushService.send(recipient: "device-token-xyz", message: "Flash sale starts now!");

        // Verify Singleton: same instance returned on second call
        NotifService mailService2 = NotifFactory.getService(type: "MAIL");
        System.out.println("\nSingleton check (MAIL same instance): " + (mailService == mailService2));

        NotifService smsService2 = NotifFactory.getService(type: "SMS");
        System.out.println("Singleton check (SMS same instance):  " + (smsService == smsService2));
    }
}
```

The main function

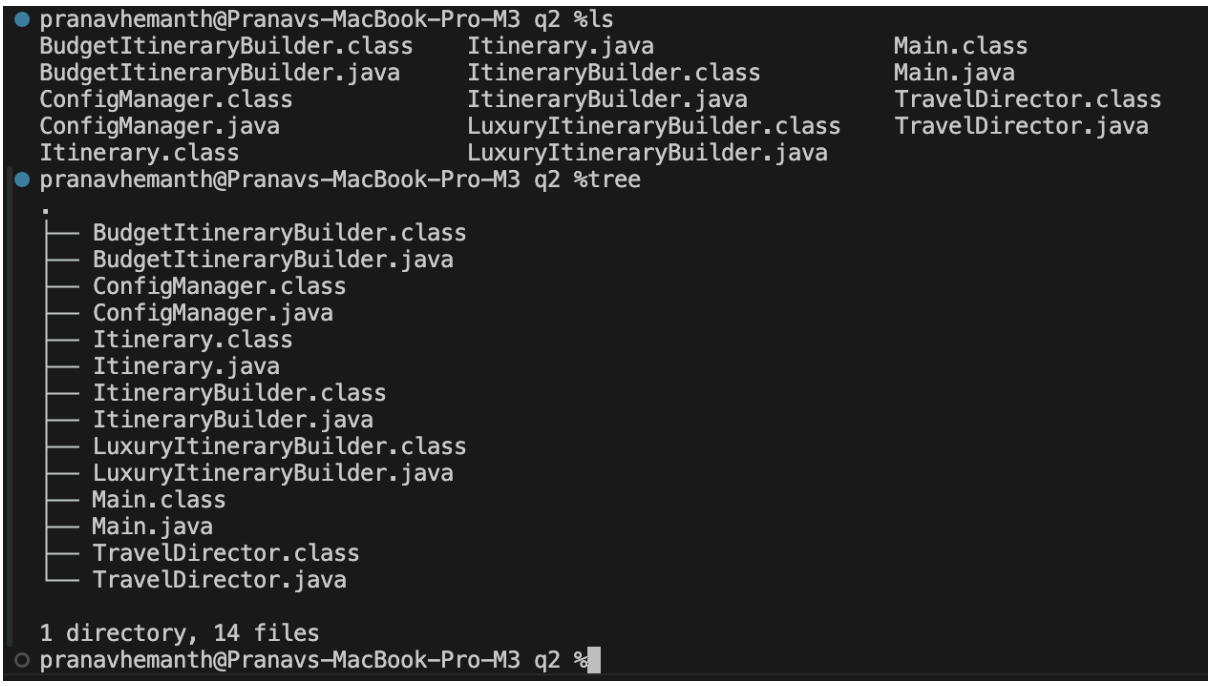
Output:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 OOAD-S6 %java Lab9.q1.Main
[MAIL] To: alice@example.com | Message: Your order has been placed.
[SMS] To: +919876543210 | Message: OTP: 482910
[PUSH] To: device-token-xyz | Message: Flash sale starts now!

Singleton check (MAIL same instance): true
Singleton check (SMS same instance):  true
pranavhemanth@Pranavs-MacBook-Pro-M3 OOAD-S6 %
```

2. You are developing a custom travel itinerary generation system similar to Make- MyTrip, where users can create personalized travel plans by selecting various components such as hotel type, transport mode, meal plan, and sightseeing options. Since each itinerary may have different combinations of these components, the object construction process becomes complex and requires step-by-step configuration.
   To address this, you are required to implement the Builder Design Pattern. The system must define a Product class (Itinerary) representing the final travel plan. An abstract Builder class (ItineraryBuilder) should declare methods for building different parts of the itinerary. Multiple Concrete Builder classes (e.g., LuxuryItineraryBuilder, Bud- getItineraryBuilder) must implement these methods to construct specific types of itineraries.
   A Director class (TravelDirector) should be responsible for orchestrating the construction process by invoking builder methods in a specific sequence. Additionally, a Singleton Configuration Manager must be implemented to provide global settings such as default budget limits, travel policies, and discount rules. All builders must access this shared instance instead of creating new configuration objects.
   Requirements:
   - Design Product, Builder, Concrete Builder, and Director classes
   - Implement a Singleton Configuration Manager
   - Demonstrate construction of at least two types of itineraries
   - Ensure no direct object creation of configuration using new

Source files list:



Project structure for Q2 showing the Product (Itinerary), abstract Builder, two Concrete Builders (Luxury/Budget), Director (TravelDirector), Singleton ConfigManager, and Main.

ConfigManager.java

Lab9 > q2 > ConfigManager.java > { } Lab9.q2

```java
package Lab9.q2;

// Singleton Config Manager — no direct instantiation via new allowed outside this class
public class ConfigManager {
    private static ConfigManager instance;

    private double defaultBudgetLimit;
    private String travelPolicy;
    private double discountRate;

    private ConfigManager() {
        // Default global settings
        this.defaultBudgetLimit = 100000.0;
        this.travelPolicy = "Standard travel insurance required for all itineraries.";
        this.discountRate = 0.10; // 10% discount
    }

    public static ConfigManager getInstance() {
        if (instance == null) {
            instance = new ConfigManager();
        }
        return instance;
    }

    public double getDefaultBudgetLimit() { return defaultBudgetLimit; }
    public String getTravelPolicy()       { return travelPolicy; }
    public double getDiscountRate()       { return discountRate; }

    @Override
    public String toString() {
        return "ConfigManager{budgetLimit=" + defaultBudgetLimit
                + ", policy='" + travelPolicy + "', discount=" + (discountRate * 100) + "%}";
    }
}
```

ConfigManager is the Singleton configuration class holding global settings (budget limit, travel policy, discount rate). The private constructor ensures no direct instantiation via new.

Itinerary.java

Lab9 > q2 > Itinerary.java > { } Lab9.q2

```java
package Lab9.q2;

// Product class
public class Itinerary {
    private String hotelType;
    private String transportMode;
    private String mealPlan;
    private String sightseeingOptions;
    private double totalCost;

    public void setHotelType(String hotelType)             { this.hotelType = hotelType; }
    public void setTransportMode(String transportMode)     { this.transportMode = transportMode; }
    public void setMealPlan(String mealPlan)               { this.mealPlan = mealPlan; }
    public void setSightseeingOptions(String options)      { this.sightseeingOptions = options; }
    public void setTotalCost(double totalCost)             { this.totalCost = totalCost; }

    public void show() {
        System.out.println(x: "--- Itinerary ---");
        System.out.println("Hotel       : " + hotelType);
        System.out.println("Transport   : " + transportMode);
        System.out.println("Meal Plan   : " + mealPlan);
        System.out.println("Sightseeing : " + sightseeingOptions);
        System.out.printf (format: "Total Cost  : Rs. %.2f%n", totalCost);
        System.out.println(x: "-----------------");
    }
}

```

Itinerary is the Product class. It holds all travel plan components (hotel, transport, meal plan, sightseeing, cost) and exposes setters that the concrete builders call to assemble the final object.

ItineraryBuilder.java

Lab9 > q2 > ItineraryBuilder.java > { } Lab9.q2

```java
// Abstract Builder
public abstract class ItineraryBuilder {
    protected Itinerary itinerary;
    // All builders access config via Singleton — no new ConfigManager()
    protected ConfigManager config = ConfigManager.getInstance();

    public void createNewItinerary() {
        itinerary = new Itinerary();
    }

    public abstract void buildHotel();
    public abstract void buildTransport();
    public abstract void buildMealPlan();
    public abstract void buildSightseeing();
    public abstract void buildCost();

    public Itinerary getItinerary() {
        return itinerary;
    }
}

```

ItineraryBuilder is the abstract builder declaring build steps; all concrete builders inherit the shared config reference via ConfigManager.getInstance() instead of creating new config objects.

LuxuryItineraryBuilder.java

Lab9 > q2 > J LuxuryItineraryBuilder.java > { } Lab9.q2

```java
package Lab9.q2;

// Concrete Builder: Luxury
public class LuxuryItineraryBuilder extends ItineraryBuilder {

    @Override
    public void buildHotel() {
        itinerary.setHotelType(hotelType: "5-Star Resort");
    }

    @Override
    public void buildTransport() {
        itinerary.setTransportMode(transportMode: "Private Jet + Limousine");
    }

    @Override
    public void buildMealPlan() {
        itinerary.setMealPlan(mealPlan: "All-Inclusive Gourmet Dining");
    }

    @Override
    public void buildSightseeing() {
        itinerary.setSightseeingOptions(options: "Private guided tours + helicopter rides");
    }

    @Override
    public void buildCost() {
        double base = 80000.0;
        // Apply discount from Singleton config — no new ConfigManager()
        double discount = base * config.getDiscountRate();
        itinerary.setTotalCost(base - discount);
    }
}

```

BudgetItineraryBuilder.java

Lab9 > q2 > J BudgetItineraryBuilder.java > { } Lab9.q2

```java
package Lab9.q2;

// Concrete Builder: Budget
public class BudgetItineraryBuilder extends ItineraryBuilder {

    @Override
    public void buildHotel() {
        itinerary.setHotelType(hotelType: "2-Star Hostel / Budget Hotel");
    }

    @Override
    public void buildTransport() {
        itinerary.setTransportMode(transportMode: "Train + Local Bus");
    }

    @Override
    public void buildMealPlan() {
        itinerary.setMealPlan(mealPlan: "Breakfast Only");
    }

    @Override
    public void buildSightseeing() {
        itinerary.setSightseeingOptions(options: "Self-guided tours + public attractions");
    }

    @Override
    public void buildCost() {
        double base = 15000.0;
        // Apply discount from Singleton config — no new ConfigManager()
        double discount = base * config.getDiscountRate();
        itinerary.setTotalCost(base - discount);
    }
}

```

LuxuryItineraryBuilder and BudgetItineraryBuilder are concrete builders that implement each build step differently, both reading the discount rate from the shared Singleton ConfigManager.

TravelDirector.java

Lab9 > q2 > TravelDirector.java > { } Lab9.q2

```java
package Lab9.q2;

// Director: orchestrates the construction sequence
public class TravelDirector {
    private ItineraryBuilder builder;

    public void setBuilder(ItineraryBuilder builder) {
        this.builder = builder;
    }

    public void constructItinerary() {
        builder.createNewItinerary();
        builder.buildHotel();
        builder.buildTransport();
        builder.buildMealPlan();
        builder.buildSightseeing();
        builder.buildCost();
    }

    public Itinerary getItinerary() {
        return builder.getItinerary();
    }
}

```

TravelDirector orchestrates the fixed build sequence. First createNewItinerary → buildHotel → buildTransport → buildMealPlan → buildSightseeing → buildCost — decoupling construction logic from the client.

Main.java

Lab9 > q2 > Main.java > { } Lab9.q2

```java
package Lab9.q2;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        // Access Singleton config (no new ConfigManager() anywhere)
        ConfigManager config = ConfigManager.getInstance();
        System.out.println("Global Config: " + config);
        System.out.println();

        TravelDirector director = new TravelDirector();

        // Build Luxury Itinerary
        System.out.println(x: "=== Luxury Itinerary ===");
        ItineraryBuilder luxuryBuilder = new LuxuryItineraryBuilder();
        director.setBuilder(luxuryBuilder);
        director.constructItinerary();
        director.getItinerary().show();

        // Build Budget Itinerary
        System.out.println(x: "=== Budget Itinerary ===");
        ItineraryBuilder budgetBuilder = new BudgetItineraryBuilder();
        director.setBuilder(budgetBuilder);
        director.constructItinerary();
        director.getItinerary().show();

        // Verify Singleton ConfigManager
        ConfigManager config2 = ConfigManager.getInstance();
        System.out.println("Singleton check (same ConfigManager): " + (config == config2));
    }
}

```

The main function

Output:

```
pranavhemanth@Pranavs-MacBook-Pro-M3 OOAD-S6 %clear
pranavhemanth@Pranavs-MacBook-Pro-M3 OOAD-S6 %java Lab9.q2.Main
Global Config: ConfigManager{budgetLimit=100000.0, policy='Standard travel insurance required for all itineraries.', discount=10.0%}

=== Luxury Itinerary ===
--- Itinerary ---
Hotel       : 5-Star Resort
Transport   : Private Jet + Limousine
Meal Plan   : All-Inclusive Gourmet Dining
Sightseeing : Private guided tours + helicopter rides
Total Cost  : Rs. 72000.00
-----------------
=== Budget Itinerary ===
--- Itinerary ---
Hotel       : 2-Star Hostel / Budget Hotel
Transport   : Train + Local Bus
Meal Plan   : Breakfast Only
Sightseeing : Self-guided tours + public attractions
Total Cost  : Rs. 13500.00
-----------------
Singleton check (same ConfigManager): true
pranavhemanth@Pranavs-MacBook-Pro-M3 OOAD-S6 %
```